



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure
Environment
Sensors



2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width`, `self.height`, `self.agent_pos`, `self.walls`, `self.food_positions`, `self.toxic_traps`, and `self.opponents`

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent environment because the `self.opponents` variable represents additional agents within the environment. These opponent agents can change the environment state and influence the main agent's actions and performance

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

sequence of every single information collected by the agent from the environment from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

sensor

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable because the `get_percept()` method does not provide the agent with the complete state of the environment. even though the agent can observe information such as its position, detected food, walls, toxins, and opponents, but it

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

measure external results because an agent should improve its performance in the real environment. Penalizing actions like hitting walls helps the agent make better decisions. Measuring internal effort alone may allow the agent to achieve a high

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding hidden toxic traps makes the environment Partially Observable because the agent cannot sense or know the trap locations. It must act based only on the limited information provided by its sensors

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The smells_toxin sensor improves rational decision-making by allowing the agent to detect hazards and avoid harmful actions. This helps the agent maximize its expected utility by achieving a higher score and reducing penalties

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit showed that an agent can exploit a faulty performance measure by maximizing the score without completing the actual task. A good metric should reward real-world success, not behaviors that only increase the score

Finalize and Lock Lab 01 Analytical Evaluation

Incomplete: 10/11 questions answered. Please provide detailed responses for all questions.

